

Programação Orientada a Objetos

Capítulo 4: Instruções de Controle

Ruan Pablio Barbosa Claudino



Objetivos da aula

1. Compreender algoritmos

O que é um algoritmo, o papel do procedimento e por que a ordem das ações determina o resultado.

2. Diferenciar as estruturas de controle

Sequência, seleção e repetição: os três blocos de construção de qualquer programa estruturado.

3. Aplicar a instrução if

Como tomar uma decisão simples, executando uma ação apenas quando uma condição é verdadeira.

4. Aplicar a instrução if...else

Como escolher entre dois caminhos possíveis e evitar armadilhas como o problema do else oscilante.

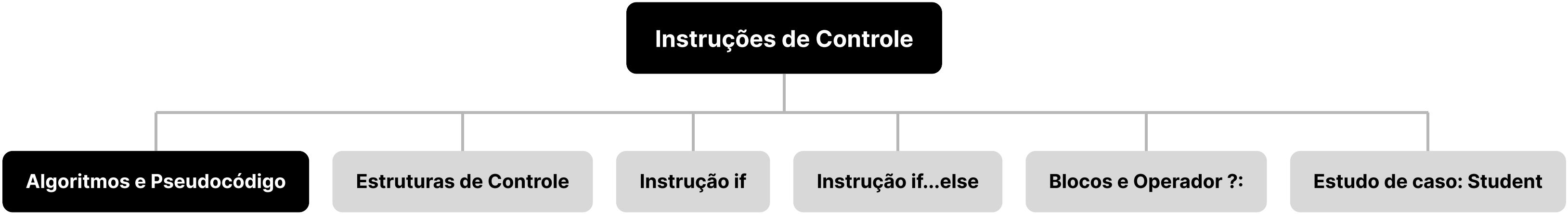
5. Usar blocos e o operador ?:

Agrupar instruções com chaves e simplificar decisões simples com o operador condicional.

Introdução

"Antes de escrever um programa para resolver um problema, você deve ter um **entendimento completo do problema** e uma **abordagem cuidadosamente planejada** para resolvê-lo."

Mapa da Aula



Algoritmos

Qualquer problema de computação pode ser resolvido executando uma série de **ações** em uma **ordem específica**. Um procedimento para resolver um problema em termos das ações a executar e da ordem em que essas ações executam é chamado **algoritmo**.

Ordem certa

1. Levantar da cama
2. Tirar o pijama
3. Tomar banho
4. Vestir-se
5. Tomar café da manhã
6. Dirigir o carro até o trabalho

Resultado: chega ao trabalho bem preparado.

Ordem errada

1. Levantar da cama
2. Vestir-se
3. Tirar o pijama
4. Tomar banho
5. Tomar café da manhã
6. Dirigir o carro até o trabalho

Resultado: chega ao trabalho completamente molhado.

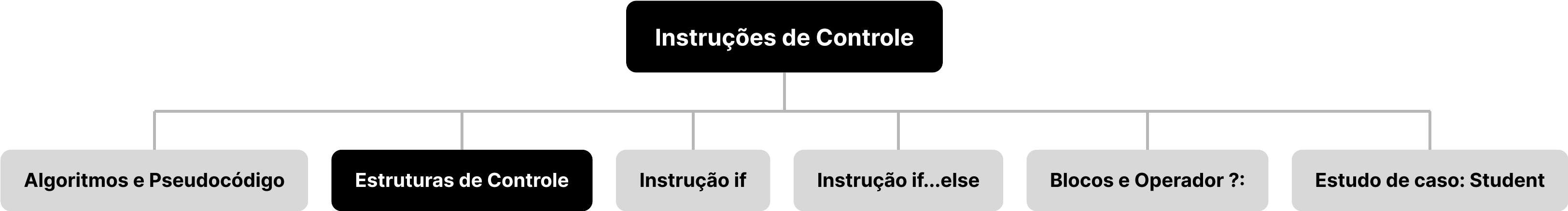
Pseudocódigo

"Pseudocódigo é uma **linguagem informal** que ajuda a desenvolver algoritmos sem se preocupar com os estritos detalhes da sintaxe da linguagem Java. **Não é executado nos computadores** — ajuda a 'estudar' um programa antes de tentar escrevê-lo em uma linguagem como Java."

Pergunta

No exemplo do executivo, trocar a ordem dos mesmos seis passos faz com que ele chegue ao trabalho **completamente molhado**. O que isso revela sobre a diferença entre escolher as ações certas e **ordená-las corretamente**?

Mapa da Aula



Estruturas de Controle

Normalmente, as instruções de um programa são executadas uma após a outra, na ordem em que são escritas — isso é chamado **execução sequencial**. Especificar uma ordem diferente é chamado **transferência de controle**.

As Três Estruturas de Controle

O trabalho de Bohm e Jacopini demonstrou que **todos os programas podem ser escritos** usando apenas essas três estruturas, combinadas de apenas duas maneiras: **empilhamento e aninhamento**.

Sequência

As instruções são executadas uma após a outra, na ordem em que aparecem no código.

Seleção

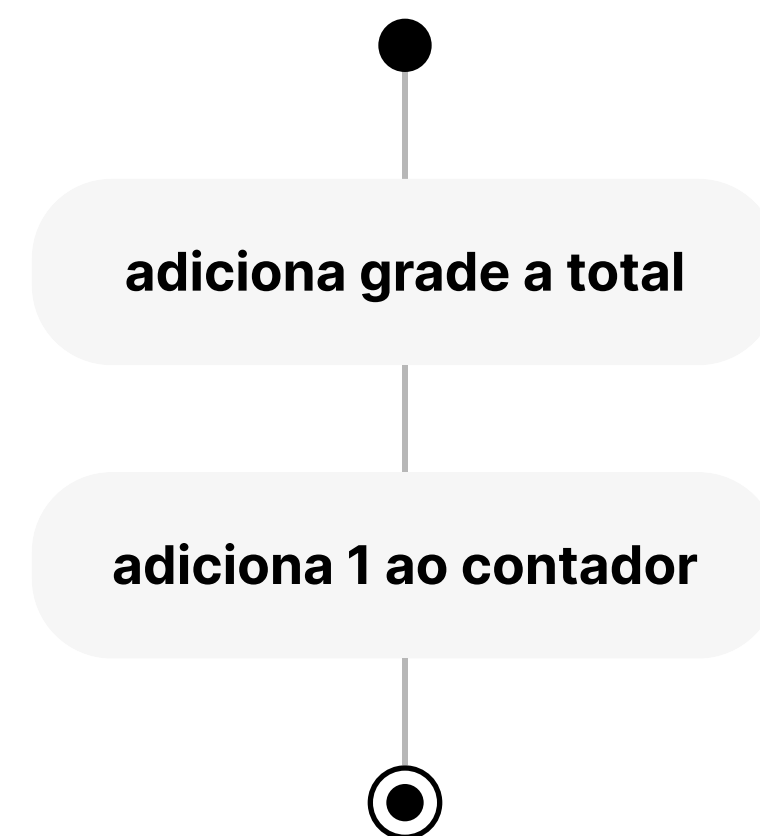
if, if...else e switch escolhem entre ações diferentes, dependendo do valor de uma condição.

Repetição

while, do...while e for repetem uma ação enquanto uma condição permanecer verdadeira.

Estrutura de Sequência

Um **diagrama de atividades UML** modela o fluxo de trabalho de um algoritmo. Símbolos do estado da ação, losangos e pequenos círculos são conectados por **setas de transição**, que representam a ordem em que as ações devem ocorrer.



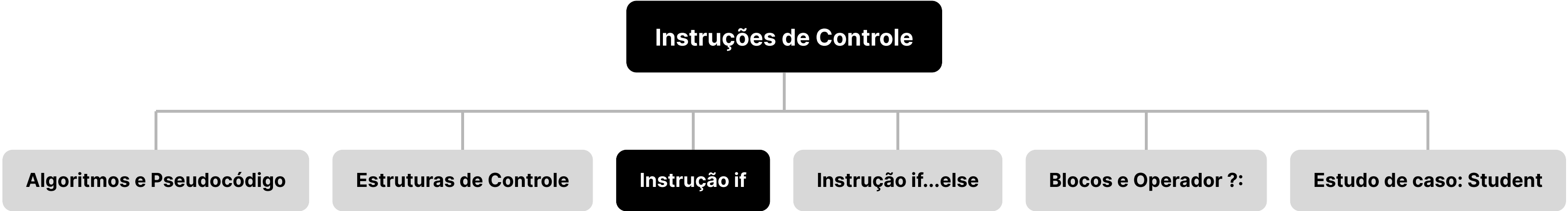
Instrução Java correspondente:
total = total + grade;

Instrução Java correspondente:
counter = counter + 1;

Pergunta

O Java não possui a instrução goto. Por que a **eliminação do goto** foi considerada um avanço tão importante para tornar os programas mais fáceis de entender, depurar e modificar?

Mapa da Aula



Instrução if: Seleção Única

A instrução if realiza uma ação apenas quando a condição é verdadeira; caso contrário, a ação é ignorada. É chamada de instrução de **seleção única** porque seleciona ou ignora uma única ação.

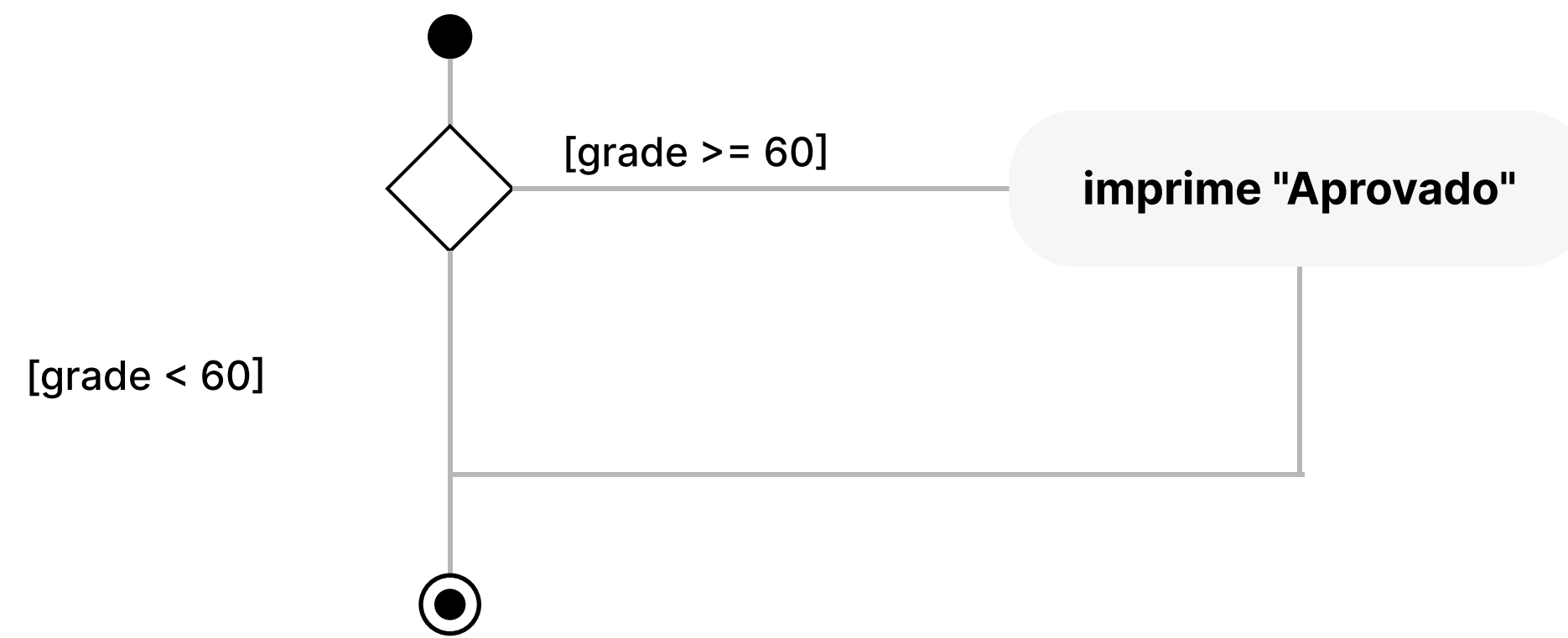
PSEUDOCÓDIGO

Se a nota do aluno for maior
que ou igual a 60
 Imprime "Aprovado"

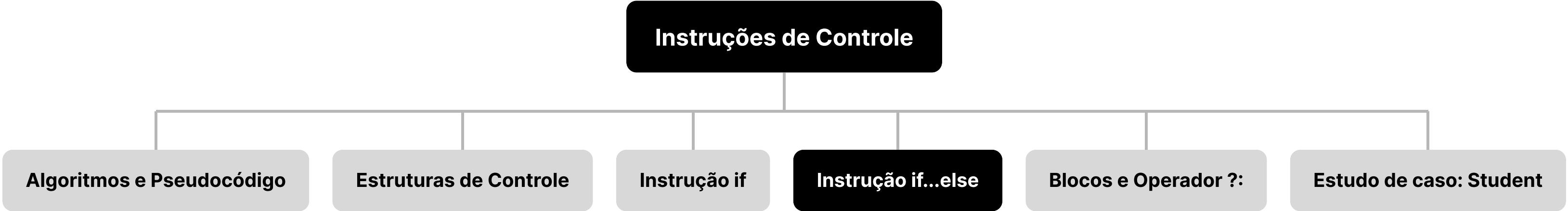
```
if (studentGrade ≥ 60)  
    System.out.println("Passed");
```


Instrução if: Diagrama de Atividades

O losango, ou **símbolo de decisão**, indica que uma decisão deve ser tomada. Cada seta de transição que sai dele tem uma **condição de guarda** — verdadeira ou falsa.



Mapa da Aula



Instrução if...else: Seleção Dupla

A instrução if...else permite especificar uma ação a realizar quando a condição é verdadeira e uma **ação diferente** quando é falsa.

PSEUDOCÓDIGO

Se a nota do aluno for maior
que ou igual a 60

 Imprime "Aprovado"

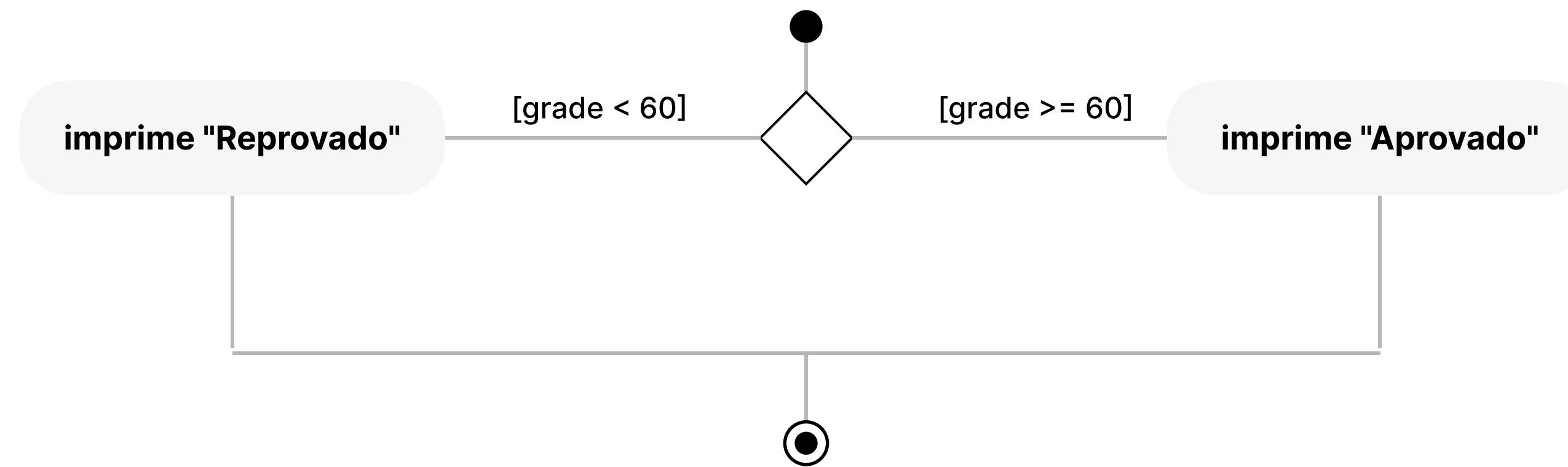
Caso contrário

 Imprime "Reprovado"

```
if (grade ≥ 60)
    System.out.println("Passed");
else
    System.out.println("Failed");
```

Instrução if...else: Diagrama de Atividades

Mais uma vez, os símbolos representam os estados e decisões da ação. Desta vez, porém, **ambas as setas de transição levam a um estado de ação** — não há caminho vazio.



if...else Aninhadas: Determinando a Nota

Um programa pode testar múltiplos casos colocando instruções if...else dentro de outras — as instruções **if...else aninhadas**. O exemplo a seguir determina a letra da nota do aluno.

```
if (studentGrade ≥ 90)
    System.out.println("A");
else
    if (studentGrade ≥ 80)
        System.out.println("B");
    else
        if (studentGrade ≥ 70)
            System.out.println("C");
        else
            if (studentGrade ≥ 60)
                System.out.println("D");
            else
                System.out.println("F");
```


Duas Formas

ANINHADA TRADICIONAL

```
if (studentGrade ≥ 90)
    System.out.println("A");
else
    if (studentGrade ≥ 80)
        System.out.println("B");
    else
        if (studentGrade ≥ 70)
            ...
```

ELSE IF (PREFERIDA)

```
if (studentGrade ≥ 90)
    System.out.println("A");
else if (studentGrade ≥ 80)
    System.out.println("B");
else if (studentGrade ≥ 70)
    System.out.println("C");
else if (studentGrade ≥ 60)
    System.out.println("D");
else
    System.out.println("F");
```

As duas formas são **idênticas**, exceto quanto ao espaçamento e recuo — o compilador ignora isso. A forma **else if** evita o recuo progressivo à direita.

O Problema do Else Oscilante

O compilador Java sempre associa um else à instrução **if imediatamente anterior**, a menos que instruído de outro modo pela colocação de chaves. Isso pode levar ao **problema do else oscilante**.

COMO PARECE

```
if (x > 5)
    if (y > 5)
        System.out.println(
            "x and y are > 5");
else
    System.out.println("x is ≤ 5");
```

COMO O COMPILADOR INTERPRETA

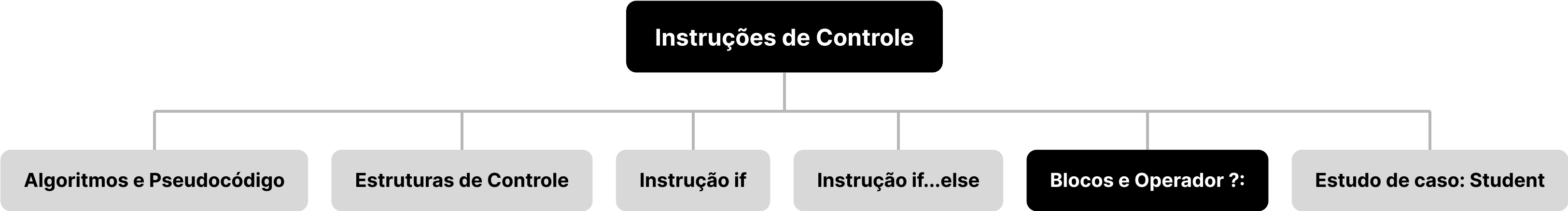
```
if (x > 5)
    if (y > 5)
        System.out.println(
            "x and y are > 5");
else
    System.out.println("x is ≤ 5");
```

Resolvendo com Chaves

As chaves indicam que a segunda instrução if está no **corpo da primeira**, e que o else está **associado à primeira instrução if**.

```
if (x > 5)
{
    if (y > 5)
        System.out.println("x and y are > 5");
}
else
    System.out.println("x is ≤ 5");
```


Mapa da Aula



Blocos

A instrução `if` normalmente espera apenas uma instrução em seu corpo. Para incluir várias instruções, envolva-as em **chaves `{ }`** — formando um **bloco**.

```
if (grade ≥ 60)
    System.out.println("Passed");
else
{
    System.out.println("Failed");
    System.out.println("You must take this course again.");
}
```

Erro de sintaxe

Capturado pelo compilador antes mesmo de o programa executar.

Erro de lógica

O programa executa, mas produz resultados incorretos.

Erro fatal de lógica

O programa falha e é encerrado prematuramente.

Operador Condicional (?:)

O Java fornece o **operador condicional (?:)**, que pode substituir uma instrução if...else simples. É o único **operador ternário** do Java — recebe três operandos.

```
System.out.println(  
    studentGrade ≥ 60 ? "Passed" : "Failed"  
);
```

CONDIÇÃO

studentGrade ≥ 60

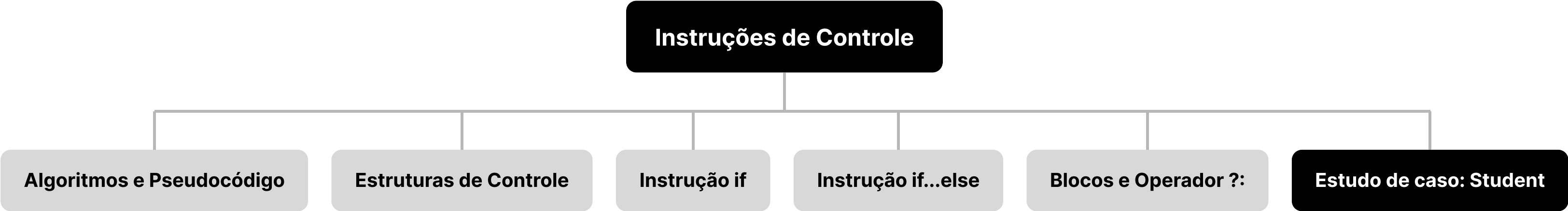
VALOR SE VERDADEIRO

"Passed"

VALOR SE FALSO

"Failed"

Mapa da Aula



Estudo de Caso: Classe Student

Atributos

name (String) e average (double) armazenam o nome e a média do aluno.

Construtor

Inicializa name e average, validando que a média esteja entre 0.0 e 100.0.

setName / getName

Definem e obtêm o nome do Student.

setAverage / getAverage

Definem e obtêm a média, com a mesma validação do construtor.

getLetterGrade

Usa if...else aninhados para converter a média em uma letra: A, B, C, D ou F.


```
// Figura 4.4: Student.java
// Classe Student que armazena o nome e a média de um aluno.
public class Student
{
    private String name;
    private double average;

    // construtor inicializa variáveis de instância
    public Student(String name, double average)
    {
        this.name = name;

        // valida que a média é > 0.0 e ≤ 100.0; caso contrário,
        // armazena o valor padrão da média da variável de instância (0.0)
        if (average > 0.0)
            if (average ≤ 100.0)
                this.average = average; // atribui à variável de instância
    }

    // define o nome de Student
    public void setName(String name)
    {
        this.name = name;
    }

    // recupera o nome de Student
    public String getName()
    {
        return name;
    }
}
```

```
// define a média de Student
public void setAverage(double studentAverage)
{
    // valida que a média é > 0.0 e ≤ 100.0; caso contrário,
    // armazena o valor atual da média da variável de instância
    if (average > 0.0)
        if (average ≤ 100.0)
            this.average = average; // atribui à variável de instância
}

// recupera a média de Student
public double getAverage()
{
    return average;
}

// determina e retorna a letra da nota de Student
public String getLetterGrade()
{
    String letterGrade = ""; // inicializado como uma String vazia

    if (average ≥ 90.0)
        letterGrade = "A";
    else if (average ≥ 80.0)
        letterGrade = "B";
    else if (average ≥ 70.0)
        letterGrade = "C";
    else if (average ≥ 60.0)
        letterGrade = "D";
    else
        letterGrade = "F";

    return letterGrade;
}
} // finaliza a classe Student
```

Classe StudentTest — Testando os Objetos

```
// Figura 4.5: StudentTest.java
// Cria e testa objetos Student.
public class StudentTest
{
    public static void main(String[] args)
    {
        Student account1 = new Student("Jane Green", 93.5);
        Student account2 = new Student("John Blue", 72.75);

        System.out.printf("%s's letter grade is: %s\n",
            account1.getName(), account1.getLetterGrade());
        System.out.printf("%s's letter grade is: %s\n",
            account2.getName(), account2.getLetterGrade());
    }
} // fim da classe StudentTest
```

SAÍDA

```
Jane Green's letter grade is: A
John Blue's letter grade is: C
```


Pergunta

Nos métodos construtor e setAverage da classe Student, por que usar **dois if simples aninhados** em vez de um único if com o operador && produz o mesmo resultado nesse caso?